



DEPARTMENT OF COMPUTER SCIENCE & ENGINEERING

Discover. Learn. Empower.

Experiment : 1

Student Name: Nakul Kumar

UID: 23BCS12354

Branch: B.E.- CSE

Section/Group: 23_Krg_2A

Semester: 6

Date of Performance: 08-01-26

Subject Name: System Design

Subject Code: 23CSH-314

1. Aim: To design, implement, and evaluate a simple URL redirection system that converts long URLs into short URLs, and to measure system performance (latency and throughput) under varying request loads.

2. Objective:

1. To understand the basic working of a URL shortening service.
2. To implement a functional URL redirection mechanism.
3. To measure and analyze latency and throughput under different simulated user loads.
4. To correlate system design choices with observed non-functional metrics (performance under load).

3. Requirements(Tools): Postman - test the api, draw.io, excalidraw, lucidcharts
, python libraries / frameworks .

4. Procedure and Output:

1. Functional Requirements :

A. Given a long url -> short url

Premium user :

A. Custom url (optional)

B. Expiry date

B. Given a short url -> redirect to the -> long url

2. Non-functional requirements :

100 million daily active user -> 1 million user are actually doing shorting of url

1. Latency: 20ms (on ur shortneing, on url redirection)

2. Availabilty: 24 x 7

3. Consistency
4. Scalability: Vertical or horizontal
5. The generated short url -> always unique

3. Api design (url shortner) :

Pre-defined functions:

1. Get: Fetch some data from db
2. Post: When you want to insert some data into db
3. Put / Patch: Update
4. Delete: Remove the data

Local host server: <https://127.0.0.1/shorten>

App.route(/)

1. Post api call - Req

```
{  
  
  Logic:  
  
  Url: "Long url "  
  
  Custom url: "Custom"  
  
  Expiry date:  
  
}
```

Res:

```
{  
  
  Short_url: "Short url "  
  
  Short_code: "123abc"  
  
}
```

<https://127.0.0.1/123abc>

2. Get api call (</short_code?)

[https://127.0.0.1/get_employee\(empid\)](https://127.0.0.1/get_employee(empid))

4. Database schema design :

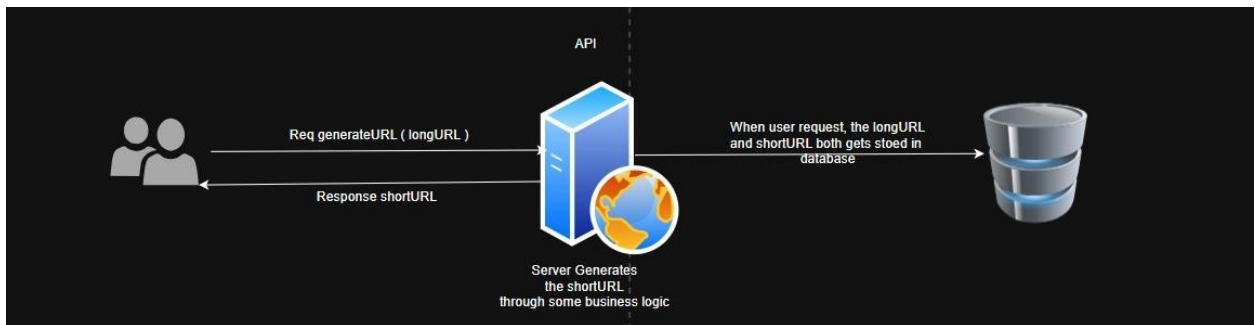
T1 user (id, name, phone etc)

T2 url_mapping (id, longurl, shorturl, expirydate, customurl)

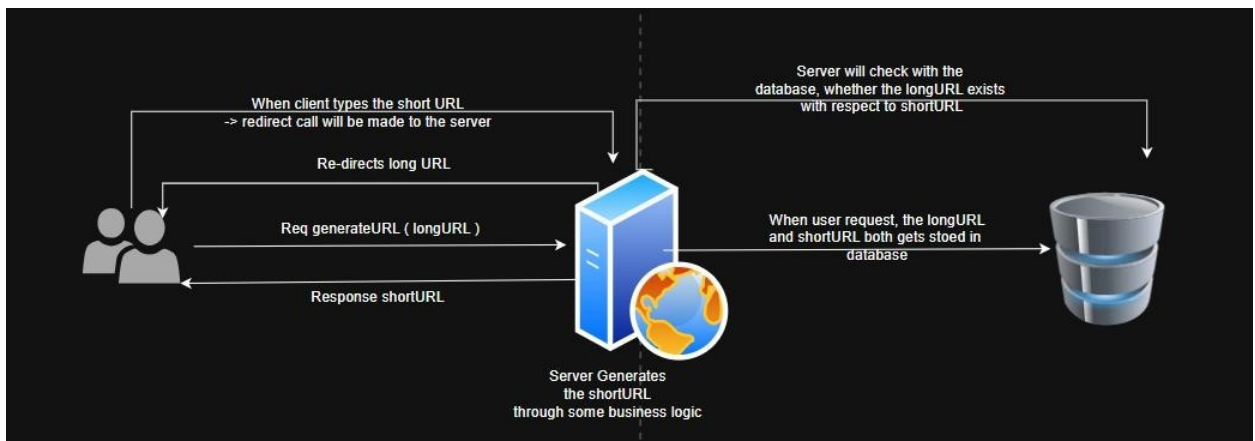
5. High Level Design :

Now According to the functional requirement of the system, we can identify that : There will be a client who is requesting, then there will a server upon which computation will be going on, and lastly there will be an database in which storage will be done.

1. shortURL Generation:



2. Re-direction: When user enters shortURL in browser:



3. We need to have REST APIs for client registration and login as well.

6. Low Level Design :

Now if you see in HLD we have only specified the specific things like the database, the server, but we haven't talked about:

1. Which type of database is needed??
2. Which type of server is needed and what sort of computation has been done inside the server.

Let's begin with the SERVER: how longURL is converted into shortURL

longURL: <https://www.amazon.com/rehan.products.catalog/orders/cart/payment>

shortURL: <https://bit.ly/5PLcymn>

{5PLcymn} - shortened URL

LONGurl - BASE10 - LARGE STRING

BASE 64 - HDGB25

Approach 01: Encryption

$\text{encrypt}(\text{longURL}) = \text{shortURL}$

Most Popular algorithm for encryption lib are: MD5, SHA1, BASE64

Problem in this approach:

1. Encrypted length is very large & the req was of short length url
2. To resolve this, what we can do is we can take first 4 letters from encrypted text. But in this also a problem can occur.

Link 1: https://www.youtube.com/watch?v=HHUi8F_qAXM&t=1s06e2116c3e064129e81226fb7b57502c

Link 2: <https://www.youtube.com/watch?v=jhjsdbi8793hch?2fdgv06e265f903040b075b90ed40ad15a4f6e5a184a2>

[both starting 4 char are same, pointing to same link in DB Duplication of links]

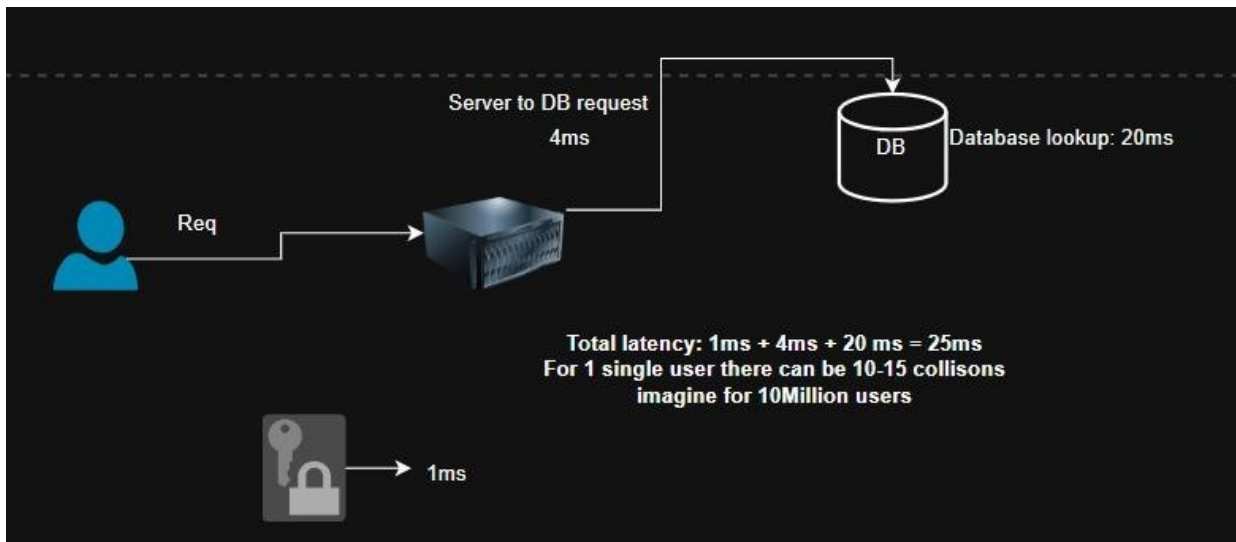
Resolution:

At first we can store the 4byte code & corresponding longURL in DB. Second time, if same 4byte code is generate, it will be compared in the DB, if code already exist -> again generate a MD5, 4byte unique code.

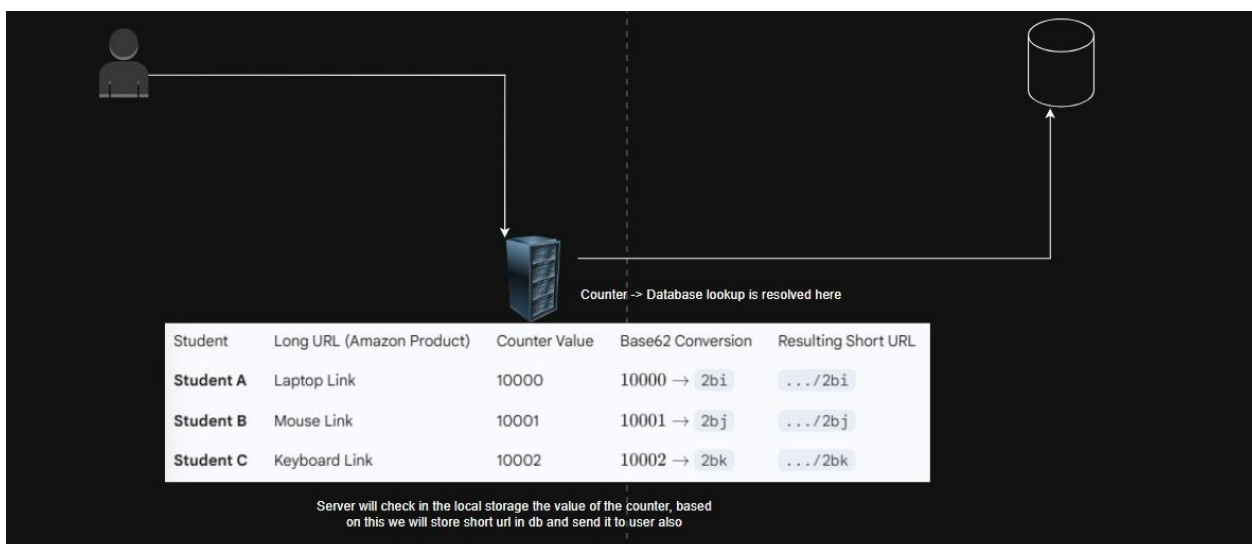
[This results into high latency issue = 16ms + 16ms]

Possible Problems:

1. Full table scan in database.
2. More number of collisions
3. These collisions will result into latency issue



Approach 02: Counter Approach

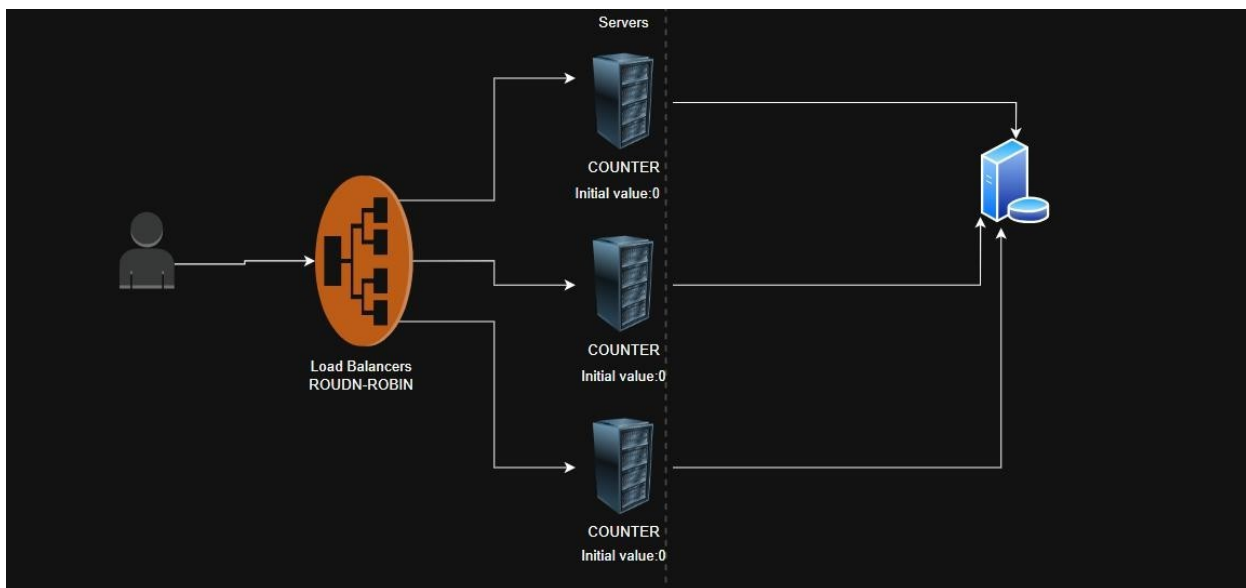
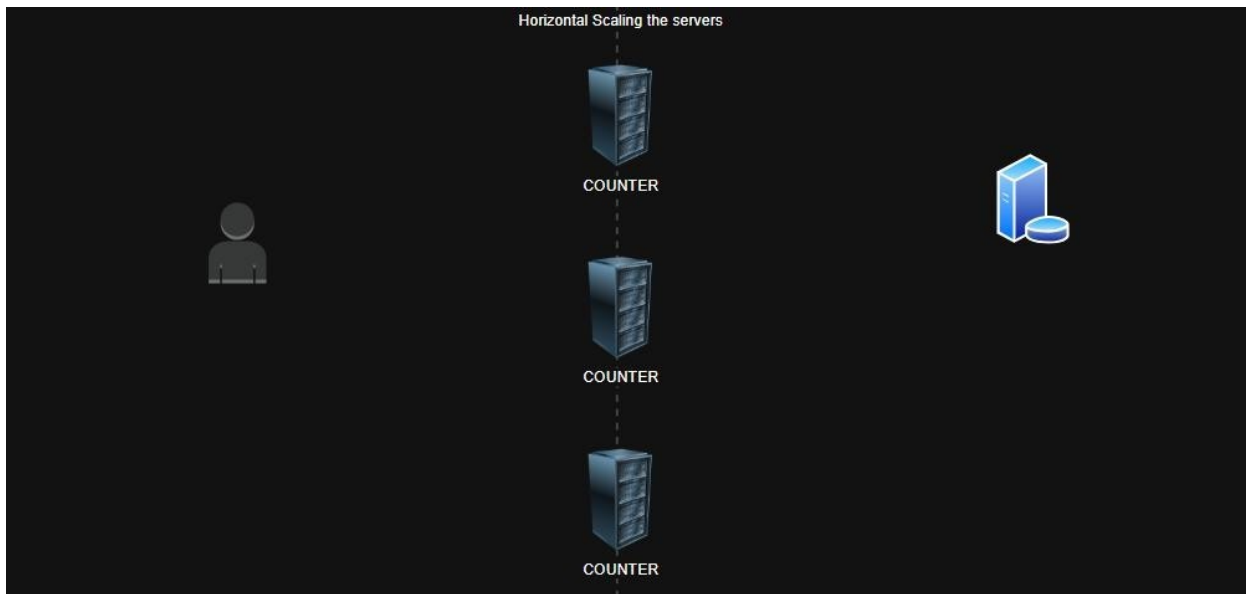


Problem with this approach:

Server follows monolithic architecture, it can process the request for 1 user easily by managing counter value, but what happens when 100 Million user comes.

Resolution: Scaling

- Vertical scaling here is not possible as number of users are 100 M
- We will do the horizontal scaling over here



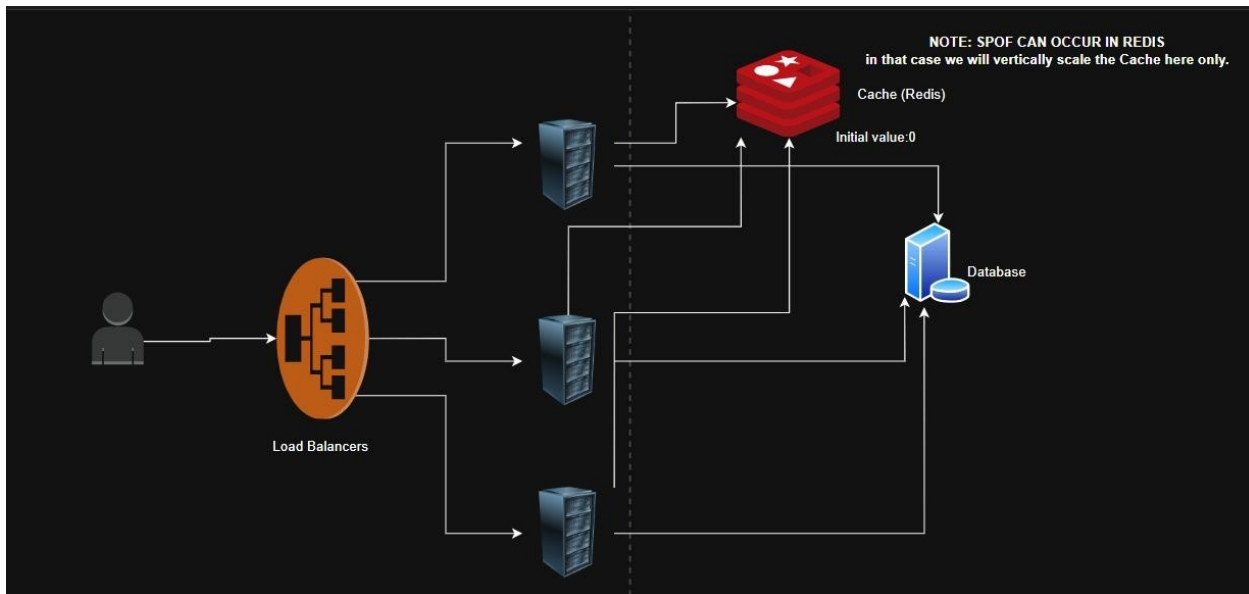
Problem :

Now in this method there can be a problem of dirty read that is server 1's counter may have some other value and server 2's counter may have some other value

Resolution:

Rather than storing the counter in the server itself, we can make the counter variable globally available using a CACHE

Final Low level Design:



5. Learning Outcomes:

- Understood and differentiated functional and non-functional requirements for designing a URL shortener system.
- Designed and tested RESTful APIs for URL shortening and redirection using Flask and Postman.
- Created high-level and low-level system designs to ensure scalability, availability, and performance.
- Designed database schemas and implemented backend logic for URL mapping and user management.